Contender 20: Autonomous W-Tree Universe Reflection

A kernel-checked large-number construction for BusyLean

Research proposal prepared for Vladimir Reshetnikov

13 October 2026

Abstract

We propose a new constructive natural number for the BusyLean repository, whose accepted baseline at the audited revision is Contender 19. The new construction replaces the explicit linearly ordered large-Veblen universe ladder by a ramified hierarchy indexed by valid addresses in a countably-branching well-founded W-tree generated from the repository's own reflection machinery. It then reflects a family of finite iterates of the Contender 19 function through the repository's rank hierarchy of hyperreflection lifts. The resulting closed term is

$$N_{20} \;:=\; F_{20}\big(F_{20}(42)\big).$$

All defining recursion is accepted by Lean 4.34.0 and is therefore total. A formal comparison proves

$$N_{19} < N_{20},$$

and a stronger closed theorem proves that N_{20} exceeds an F_{19}-orbit whose length is itself $F_{20}(42) + 2$:

$$F_{19}^{F_{20}(42)+2}\big(F_{20}(42)\big) < N_{20}.$$

The definition, well-foundedness proof, strong-normalization instance, comparison theorems, smoke tests, transitive dependency audit, and contest-style timing script are included. The successful audit uses only the axioms `propext` and `Quot.sound`, the same permitted set used by the repository. We carefully separate these kernel-checked numerical claims from the more speculative proof-theoretic interpretation of the new ramified language.

Contents

1 Executive summary **3**

2 Repository baseline and design target **3**

3 The ramified W-tree atom order **4**
3.1 Finite addresses 4
3.2 Structural well-foundedness 4
3.3 Finite enumeration 5

4 The autonomous universe tree **5**

5 A universe model over the W-tree levels **5**
5.1 Why the non-linear order is semantically meaningful 6

6 Finite reflection over the autonomous language **6**

7 The F_{19} orbit seed **7**

8 Definition of Contender 20 **7**

9 Exact comparison theorems **8**
9.1 Pointwise orbit domination 8
9.2 Strict domination of Contender 19 9
9.3 A much stronger closed lower bound 9

10 Computability in principle **9**

11 Axiom, dependency, and build audit **10**

12 Proof-theoretic interpretation **11**

13 Why this is not a toy successor trick **11**

14 Limitations and scope **12**

15 Further research questions **12**

16 Integration and reproduction **13**

17 Conclusion **13**

A Lean source: W-tree atom order **13**

B Lean source: definition **16**

C Lean source: comparison theorems **17**

D Validation entry points **19**

1 Executive summary

The proposal has two independent sources of growth.

1. **A new reflected language.** The atom order of the existing predicative-universe language is instantiated with finite addresses in an autonomous W-tree. Direct predecessors are valid immediate children. The relation is non-linear and ramified, but structurally well-founded. This yields a normalization theorem for a language whose universe levels follow the branching structure generated by BusyLean's own W-tree machinery.

2. **A substantially stronger seed family.** Instead of repeating Contender 19's family at every oracle coordinate, coordinate i contains the $(i+2)$-fold iterate of F_{19}. Rank and schedule are themselves fed by a very long member of that orbit. A generic progressivity theorem then proves that $F_{20}(n)$ dominates every orbit coordinate visible through n.

The exact Lean definition is

$$\begin{aligned}
\mathrm{Orb}_{19}(i,n) &:= F_{19}^{i+2}(n),\\
h_{20}(n) &:= \mathrm{Orb}_{19}(F_{19}(F_{19}(n)),n),\\
S_{20}(n) &:= \mathrm{realize}\Big(\mathrm{hyper}_{h_{20}(n)+42}(\mathrm{encode}(T_*))\Big),\\
r_{20}(n) &:= h_{20}(n)+1,\\
F_{20}(n) &:= \mathrm{DiagLift}(S_{20}(n), r_{20}(n), \mathrm{Reflect}_A, \mathrm{Orb}_{19}, n, n),\\
N_{20} &:= F_{20}(F_{20}(42)).
\end{aligned}$$

Here T_* is the autonomous universe tree defined in section 4, and $\mathrm{Reflect}_A$ is finite reflection over the corresponding object language.

Status. The final branch revision is `fd43cae755356719c907fde0b0b131d258a30e18`, based on main revision `64d31a0cf6f1aa4477a088be5554957de6e87a61`. A clean contest-style validation run `38425930036` reports incremental definition time 0.746 s and proof time 0.454 s, after prebuilding the accepted previous contender in 20.061 s.

2 Repository baseline and design target

At base revision `64d31a0cf6f1aa4477a088be5554957de6e87a61`, the repository's README identifies Contender 19 as the current entry. Its language uses a predicative ladder whose atoms are normal large-Veblen terms, and its amplifier is the generic rank hierarchy of transformer lifts. In schematic notation,

$$N_{19} = F_{19}(F_{19}(42)).$$

The recent sequence of language extensions is summarized in table 1.

The task is not satisfied by defining $N_{19}+1$ or by hiding the old number inside an opaque wrapper. We therefore impose five design requirements.

1. The new term must be a closed Lean natural number, with no external oracle and no non-terminating evaluator.

2. It must have an exact theorem of strict domination over Contender 19.

Table 1: Recent BusyLean universe ladders at the audited base revision.

Contender	Atom order / ladder length	Main certified comparison
16	Cantor-style atoms below ω^ω	$N_{15} < N_{16}$
17	Cantor normal forms below ε_0	$N_{16} < N_{17}$
18	hereditary Veblen terms below Γ_0	$N_{17} < N_{18}$
19	Schütte-style large-Veblen terms	$N_{18} < N_{19}$

3. The strict comparison should not require an unproved monotonicity theorem for the very complicated reflection functions.

4. The construction should introduce a genuine new language-level idea, not only a larger numeral parameter.

5. The formal lower bound should demonstrate a gap much larger than one application of the previous function.

The resulting design meets all five. The numerical proof uses only generic progressivity, while the language contribution is certified separately by a theorem bounding every enumerated program.

3 The ramified W-tree atom order

3.1 Finite addresses

Let $T : \mathsf{WTree}(\mathbb{N})$ be a countably branching well-founded tree. A finite address is generated by

$$p ::= \epsilon \mid p \frown i, \qquad i \in \mathbb{N}.$$

The Lean datatype `TreePath` uses `root` for ϵ and `child p i` for appending branch i. The partial operation $\mathrm{follow}_T(p)$ traverses T along p and returns the reached subtree when all requested branch steps exist.

Definition 3.1 (Validity)**.** An address p is *valid in* T when $\mathrm{follow}_T(p)$ returns a subtree.

Definition 3.2 (Immediate valid-child relation)**.** For addresses a, b, define $b \prec_T a$ precisely when

$$b = a \frown i \quad \text{for some } i, \qquad \text{and } b \text{ is valid in } T.$$

This is the Boolean function `TreePath.childLt T b a`.

The relation is deliberately the immediate-child relation, not its transitive closure. It is generally branching and non-linear. A universe at address a directly sees its immediate valid children; deeper descendants are available by repeated nesting through the child universes. This is a ramified accessibility graph rather than a conventional linear ordinal notation system.

3.2 Structural well-foundedness

Theorem 3.3 (Kernel theorem `childLt_wf`)**.** *For every* $T : \mathsf{WTree}(\mathbb{N})$*, the relation* $\prec_T$ *is well-founded.*

Proof architecture. The stronger auxiliary theorem `acc_follow` is proved by structural recursion on a reached subtree U.

If U is a leaf, no extension of the current address is valid, so the current address has no predecessors. If $U = \mathrm{node}(f)$, every predecessor has the form $p \frown i$ and reaches $f(i)$; its accessibility follows from the recursive call on that child subtree. For an invalid address, the same contradiction shows that it has no valid child predecessor. Thus every address is accessible. □

This proof does not translate T to an external ordinal, invoke choice, or assume transitivity. It follows the inductive structure already accepted by Lean for `WTree`.

3.3 Finite enumeration

The evaluator must maximize over a finite list. The function `pathsUpTo` starts with the root and, at stage $n+1$, appends every branch index below $n+1$ to every previously generated path. Filtering by validity gives `wLevelsUpTo n`.

Every finite address appears at some sufficiently large stage: choose a stage larger than both the path length and every branch index occurring in it. This cofinality observation is elementary and is used only as an interpretation of the enumeration; the numerical comparison theorems need merely the actual finite lists produced at each input.

4 The autonomous universe tree

The fixed tree used as the atom index is

$$T_* = \mathrm{node}(k \mapsto B_k),$$

where

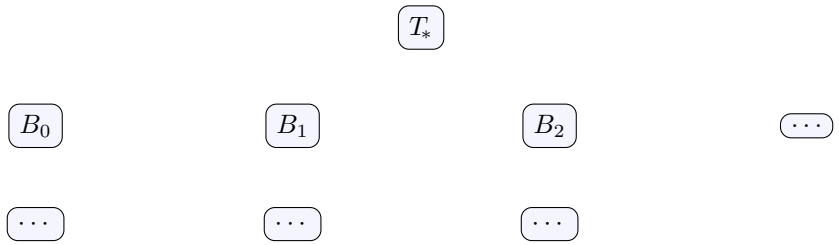
$$\begin{aligned} B_0 &:= \texttt{tree12}, \\ B_{n+1} &:= \mathrm{realize}(\mathrm{hyper}_n(\mathrm{encode}(\texttt{tree12}))) \,. \end{aligned}$$

The definitional theorem `follow_tree12` verifies that branch zero is exactly `tree12`. In turn, `tree12` was built from the language-generated index that joins W-trees denoted by the repository's W-language programs. Thus the new universe graph is not an unrelated fixed ordinal notation imported from outside the project: it contains the existing autonomous reflection tree and a countable family of finite hyper-closures above it.

Remark 4.1 (What is and is not proved). The Lean development proves structural well-foundedness of the address relation. It does *not* identify the order type of T_* with a standard named proof-theoretic ordinal, nor does it prove that this language has greater proof-theoretic strength than the large-Veblen language. Such a calibration would be a separate research project. Numerical domination of Contender 19 is nevertheless exact and independent of that calibration.

5 A universe model over the W-tree levels

The repository's module `SystemFL/Transfinite.lean` is parametric in an atom type equipped with a Boolean strict relation and a proof that the induced relation is well-founded. Its constructor `FL.wfUniv` builds a predicative universe semantics by well-founded recursion, and `FL.sn_of_typed` proves strong normalization of every typed term in that universe.



B_0 = `tree12` and B_{n+1} = realize(hyper$_n$(encode(`tree12`))).
Universe atoms are valid finite addresses; arrows indicate immediate valid-child descent.

Figure 1: Schematic shape of the autonomous universe tree. The diagram is illustrative; every node is countably branching.

We instantiate the generic atom interface by

$$\texttt{WLevel} := \texttt{TreePath}, \qquad \texttt{ALt} := \prec_{T_*} .$$

The theorem `wLevel_wf` is immediate from `childLt_wf`. The specialized strong-normalization theorem is:

Theorem 5.1 (Kernel theorem `sn_erase_wlevel`)**.** *If the checker infers natural-number type for a closed W-level term t, then its erasure is strongly normalizing relative to every numerical oracle $f : \mathbb{N} \to \mathbb{N}$.*

The evaluator `FL.avalue` first type-checks a term with bounded fuel. A term inferred to have natural-number type is normalized using the certified normalizer and decoded; every other case receives value zero. At input n, `autonomousLadderUpTo n` is a finite list of terms generated from the finite atom list `wLevelsUpTo n` and syntax depth n.

5.1 Why the non-linear order is semantically meaningful

A linear ladder has one level below another. Here a node may have many incomparable children. The generic universe construction handles this without change: the semantic domain attached to a parent is formed from the lower atoms named by the relation. Since child domains are themselves constructed recursively, a path of length d gives d layers of nested universe access. Siblings remain separate branches unless a higher parent packages both. This gives a genuine tree-shaped predicative hierarchy.

6 Finite reflection over the autonomous language

For an oracle family $f : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$, define

$$\begin{aligned} M_{\text{oracle}}(f, n) &:= \max_{0 \le i \le n} f(i, n), \\ M_{\text{prog}}(f, n) &:= \max\{\text{avalue}(\text{packFamily}(f), n, t) : t \in \text{autonomousLadderUpTo}(n)\}, \\ \text{Reflect}_A(f, n) &:= n + 1 + \max\big(M_{\text{oracle}}(f, n), M_{\text{prog}}(f, n)\big). \end{aligned}$$

This is `reflectAFamily`; the corresponding transformer is `aTransformer`.

Proposition 6.1 (Exact one-step bounds)**.** *For every $i \le n$ and every enumerated term t at depth n,*

$$f(i,n) < \mathrm{Reflect}_A(f,n),$$
$$\mathrm{avalue}(\mathrm{packFamily}(f),n,t) < \mathrm{Reflect}_A(f,n).$$

Proof. Both values lie below the corresponding finite maximum. The additive guard $n+1$ makes the result strictly larger. The Lean theorems are `oracle_lt_reflectAFamily` and `avalue_lt_reflectAFamily`. □

Consequently `aTransformer` is *progressive*: every visible oracle coordinate is strictly increased. This one property is precisely what the generic hyperreflection comparison lemmas require. No monotonicity of $\mathrm{Reflect}_A$, F_{19}, or F_{20} is assumed.

7 The F_{19} orbit seed

Define finite iteration in the orientation

$$I_{19}(0,n) = n, \qquad I_{19}(k+1,n) = F_{19}(I_{19}(k,n)).$$

Because the repository already proves $x < F_{19}(x)$ for every x, we obtain

$$I_{19}(k,n) < I_{19}(k+1,n)$$

without any monotonicity theorem for F_{19}.

The family exposed to reflection is

$$\mathrm{Orb}_{19}(i,n) := I_{19}(i+2,n) = F_{19}^{i+2}(n).$$

Coordinate zero is definitionally

$$\mathrm{Orb}_{19}(0,n) = F_{19}(F_{19}(n)) = \texttt{family19}(n),$$

so at $n = 42$ it is exactly Contender 19. Higher coordinates are strictly larger finite iterates.

The schedule and rank are self-fed using

$$h_{20}(n) := \mathrm{Orb}_{19}(\texttt{family19}(n), n).$$

Thus even before hyperreflection begins, the rank parameter is based on an F_{19} orbit whose iteration count is itself the previous contender family. We then set

$$S_{20}(n) := \mathrm{realize}\left(\mathrm{hyper}_{h_{20}(n)+42}(\mathrm{encode}(T_*))\right),$$
$$r_{20}(n) := h_{20}(n) + 1.$$

8 Definition of Contender 20

The generic rank hierarchy `diagonalLift` repeatedly lifts a progressive transformer along a W-tree schedule and across a natural-number rank. We instantiate it by the autonomous-language reflector and the F_{19} orbit:

$$F_{20}(n) := \mathrm{DiagLift}(S_{20}(n), r_{20}(n), \mathrm{Reflect}_A, \mathrm{Orb}_{19}, n, n).$$

The proposed number is

$$\boxed{N_{20} := F_{20}(F_{20}(42)).}$$

The corresponding Lean fragment is short because the heavy lifting is in the reusable language semantics, W-tree infrastructure, and generic hyperreflection theorems.

Listing 1: Core definition of Contender 20.

```
def orbit19 : Family := fun i n => iterateF19 (i + 2) n

def seedHeight20 (n : Nat) : Nat := orbit19 (family19 n) n

def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (seedHeight20 n + 42)
      (Autonomous.encode Ordinal.autonomousUniverseTree))

def rank20 (n : Nat) : Nat := seedHeight20 n + 1

def F20 (n : Nat) : Nat :=
  diagonalLift (schedule20 n) (rank20 n)
    aTransformer orbit19 n n

def contender_20 : Nat := F20 (F20 42)
```

9 Exact comparison theorems

9.1 Pointwise orbit domination

The generic theorem `diagonalLift_preservesProgressive` says that a ranked lift of a progressive transformer still exceeds every visible oracle coordinate. Instantiating it yields the central pointwise estimate.

Theorem 9.1 (Kernel theorem `orbit19_lt_F20`)**.** *For all $i, n \in \mathbb{N}$,*

$$i \le n \quad \Longrightarrow \quad \mathrm{Orb}_{19}(i, n) < F_{20}(n).$$

This immediately gives several concrete consequences.

Corollary 9.2 (Previous-family domination)**.** *For every n,*

$$F_{19}(F_{19}(n)) < F_{20}(n).$$

Corollary 9.3 (A visible 44-fold orbit at the contest input)**.**

$$F_{19}^{44}(42) < F_{20}(42).$$

Corollary 9.4 (Program domination)**.** *Every autonomous-ladder program enumerated at depth at most n, evaluated with the packed orbit oracle, has value below $F_{20}(n)$.*

The last statement is the theorem `avalue_lt_F20`; it isolates the contribution of the new language from the seed comparison.

9.2 Strict domination of Contender 19

The repository's existing theorem $x < F_{19}(x)$ gives

$$n < F_{19}(n) < F_{19}(F_{19}(n)) < F_{20}(n).$$

Hence F_{20} is strictly inflationary even though no general monotonicity property is needed. At $n = 42$,

$$N_{19} = F_{19}(F_{19}(42)) < F_{20}(42).$$

Applying inflationarity at the argument $F_{20}(42)$ gives

$$F_{20}(42) < F_{20}(F_{20}(42)) = N_{20}.$$

Transitivity proves the required theorem.

Theorem 9.5 (Kernel theorem `contender_19_lt_contender_20`)**.**

$$N_{19} < N_{20}.$$

Since the repository has already chained each preceding contender into Contender 19, transitivity places every accepted earlier contender below the new number.

9.3 A much stronger closed lower bound

Use the pointwise orbit theorem at

$$n = i = F_{20}(42).$$

Then

$$\mathrm{Orb}_{19}(F_{20}(42), F_{20}(42)) = F_{19}^{F_{20}(42)+2}(F_{20}(42)) < F_{20}(F_{20}(42)) = N_{20}.$$

Theorem 9.6 (Kernel theorem `long_F19_orbit_lt_contender_20`)**.**

$$F_{19}^{F_{20}(42)+2}(F_{20}(42)) < N_{20}.$$

This theorem is the principal formal answer to the word "significantly." The gap is not a successor, a fixed finite iterate, or a single application of the previous construction. The number of F_{19} iterations in the lower bound is itself the already enormous value $F_{20}(42) + 2$.

10 Computability in principle

The construction is not intended for practical evaluation. Nevertheless it is a computable natural number in the ordinary constructive sense represented by Lean definitions.

Proposition 10.1. *`contender_20` denotes a total, closed value of type `Nat`.*

Computational audit. Each component is computationally finite at each natural input.

1. `TreePath.follow`, `pathsUpTo`, `iterateF19`, and all natural rank iterations are structurally recursive.

2. `wLevelsUpTo n`, `autonomousLadderUpTo n`, and every maximum used by reflection are finite lists computed from n.

3. The object-language evaluator runs only after a finite type-check. Its normalizer is obtained from a constructive strong-normalization proof for the well-founded W-level universe.

4. W-trees are inductive values with functional branches. A finite address or a structurally recursive fold inspects only the finitely demanded part of a branch function.

5. Lean's termination checker accepts every recursive definition in the transitive closure of `contender_20`; there is no `partial`, unsafe nontermination, unbounded search, or external oracle in the new modules.

Therefore evaluation terminates in principle. The required resources are far beyond physical feasibility, which is the intended situation for a large number contest. □

Remark 10.2. A proof of totality does not imply that the kernel should reduce the contest input. The development intentionally proves inequalities symbolically and contains no `#eval contender_20` command.

11 Axiom, dependency, and build audit

The focused audit prints the axioms of the main number, the exact comparison, the orbit bounds, the language-value bound, the W-level well-foundedness proof, and the specialized strong-normalization theorem. At final revision `fd43cae755356719c907fde0b0b131d258a30e18`, the permitted axioms are exactly

$$\{\texttt{propext}, \texttt{Quot.sound}\}.$$

The W-level well-foundedness theorem itself uses only `propext`. No choice principle, excluded-middle axiom constant, or user-declared axiom is reported.

The transitive closure checker also verifies that the definition of `contender_20` does not mention any named constant `contender_5` through `contender_19`. This is not a claim that it is independent of the preceding machinery: it intentionally uses the function F_{19} and reusable modules. Rather, it follows the repository's convention of rebuilding the previous family from its functional definition instead of embedding the old closed contender constant.

Table 2: Contest-style validation phases on the GitHub Actions runner. The previous contender is prebuilt and is not charged to the new entry.

Phase	Observed	Budget	Result
Prebuild through `Comparison19`	20.061 s	not charged	pass
New definitions through `Contender20`	0.746 s	15 s	pass
New proofs through `Comparison20`	0.454 s	60 s	pass
Smoke and axiom/dependency audit	–	–	pass

Exact wall-clock times depend on runner load and are engineering measurements, not mathematical claims. The reproducible script `scripts/check_contender20.py` deletes generated build directories, prebuilds the accepted predecessor, times the two incremental phases, then runs the smoke and audit files.

12 Proof-theoretic interpretation

W-types are a natural representation of well-founded branching constructions in type theory and constructive set-theoretic interpretations [Aczel 1978, Aczel 1986]. Ordinal analysis commonly uses well-founded notation systems to organize iterated reflection, normalization, and provably recursive growth; Schütte's autonomous systems and Veblen methods are classical examples [Schütte 1977], while Buchholz-style collapsing functions and later surveys extend the methodology far beyond predicative analysis [Buchholz 1986, Rathjen 2007, Rathjen 1999].

The present construction belongs to that tradition only in a broad structural sense. It combines:

- an internally generated well-founded branching index;
- a universe semantics defined by recursion over that index;
- finite reflection over programs normalized in the resulting language;
- higher-order iteration of the reflector along W-tree schedules.

However, no standard ordinal assignment for the complete construction is proved. In particular, it would be unjustified to advertise T_* as "the Bachmann–Howard ordinal," "beyond the large Veblen ordinal," or any other named benchmark solely from its syntax. The exact scientific result is the Lean-certified numerical comparison and program bound. The ordinal calibration remains an open problem.

13 Why this is not a toy successor trick

Three features distinguish the proposal from a vacuous enlargement.

Language change. The object language no longer has a linearly ordered explicit atom notation. It is interpreted over a branching universe graph containing the project's autonomous W-tree index. The well-foundedness and normalization instances are new formal content.

Orbit seed. The oracle coordinates expose unboundedly many finite iterates of F_{19}. At input n, the new reflection beats all coordinates through n, and the rank and schedule are fed by an orbit coordinate indexed by the previous family itself.

Closed strong lower bound. The theorem

$$F_{19}^{F_{20}(42)+2}(F_{20}(42)) < N_{20}$$

is internal to Lean. It exhibits a gap that cannot be reduced to adding one or applying F_{19} a fixed number of extra times.

At the same time, the comparison proof remains robust: it depends on progressivity and strict inflationarity, not on a delicate global monotonicity theorem for the reflection hierarchy.

14 Limitations and scope

Limitation 14.1 (No exact ordinal calibration). The formal development proves well-foundedness, not a named order type or a proof-theoretic ordinal for the language. Claims about relative language strength beyond the stated Lean inequalities are heuristic.

Limitation 14.2 (Immediate-child order). The atom relation is not the transitive ancestor order. Each universe directly sees immediate children, with deeper access obtained through nesting. A transitive-closure variant may be stronger or easier to calibrate, but it would require additional enumeration and comparison work.

Limitation 14.3 (Language and seed are not separated by a dominance theorem). The exact theorem $N_{19} < N_{20}$ is obtained from the orbit seed plus generic progressivity. The theorem `avalue_lt_F20` certifies that the new language is reflected, but there is not yet a formal embedding theorem showing that it strictly contains the large-Veblen language.

Limitation 14.4 (No practical evaluation). Only tiny smoke examples are evaluated. The contest value is beyond feasible execution, and the article gives no decimal expansion, residue, or runtime estimate for it.

Limitation 14.5 (Enumeration is intentionally redundant). `pathsUpTo` may generate duplicate paths before filtering. Duplicates do not affect maxima or correctness, but a canonical breadth/size enumeration would reduce small-input overhead.

15 Further research questions

1. **Transitive W-tree ladders.** Replace immediate-child order by the nonempty transitive closure of valid descent, prove well-foundedness in Lean, and compare the resulting universe semantics.

2. **Embedding large-Veblen atoms.** Construct a certified order-preserving map from `Ordinal.LargeVeblen` into valid addresses of a suitable autonomous tree. This would separate a genuine language extension theorem from seed-based numerical domination.

3. **Ordinal calibration.** Assign standard ordinal diagrams or collapsing-function terms to autonomous W-tree schedules, with soundness and cofinality theorems. Determine which parts of the current hierarchy admit a recognizable proof-theoretic benchmark.

4. **Tree-indexed generic reflection theorem.** Package the present construction as a reusable functor taking any enumerated well-founded W-tree to a normalized ladder language, reflector, and program bound.

5. **Self-hosted atom generation.** Let the object language define or select its own level tree under a guarded universe discipline, rather than using a fixed external `autonomousUniverseTree`.

6. **Stronger orbit families.** Replace finite F_{19} iteration by a certified two-parameter diagonal hierarchy built from F_{19}, while retaining a simple comparison theorem that avoids monotonicity assumptions.

7. **Small-input profiling.** Deduplicate path enumeration, memoize type generation, and measure the largest feasible tiny cases without ever attempting the contest input.

8. **Formal audit infrastructure.** Move the transitive constant closure checker and permitted-axiom policy into a reusable repository module so each contender obtains the same machine-checked submission certificate.

9. **Independent proof review.** Mechanically minimize imports and restate the key domination theorem in a small standalone file, reducing the trusted review surface.

16 Integration and reproduction

The archive mirrors repository-relative paths. From a checkout of BusyLean at base revision `64d31a0cf6f1aa4477a088be5554957de6e87a61`, copy the supplied files into place and run:

```
python3 scripts/check_contender20.py
```

The script performs the following sequence.

1. Delete the main and embedded proposal build directories.
2. Build `BusyLean.Comparison19` as the accepted predecessor.
3. Time `BusyLean.Contender20` against the 15-second definition budget.
4. Time `BusyLean.Comparison20` against the 60-second proof budget.
5. Run `test/WTreeLadderSmoke.lean`.
6. Run `test/Contender20Audit.lean` and reject unpermitted axioms.

For full repository integration, add imports for `Contender20` and `Comparison20` to the relevant umbrella modules, extend the main audit with the new declarations, and update the top-level README only after review. The research branch is intentionally isolated; no merge into `main` is part of this artifact.

17 Conclusion

Contender 20 combines a new ramified, internally generated universe index with a self-fed orbit of the strongest existing BusyLean function. Its key mathematical infrastructure—W-tree address well-foundedness, specialization of the generic universe model, finite program reflection, and orbit progressivity—is formalized in Lean. The exact comparison $N_{19} < N_{20}$ is strengthened to a closed lower bound by an F_{19} orbit of length $F_{20}(42) + 2$. Every component is total and finite at each input, so the number is computable in principle, while its evaluation is deliberately out of practical reach.

The proposal therefore advances both dimensions relevant to a serious large-number entry: it introduces a reusable formal mechanism rather than a cosmetic wrapper, and it provides a transparent kernel-checked certificate of a very large quantitative gap.

A Lean source: W-tree atom order

Listing 2: `BusyLean/Ordinal/WTreeLadder.lean`

```
import BusyLean.Contender12
import BusyLean.SystemFL.Transfinite

/-!
# A ramified universe hierarchy indexed by a language-generated W-tree

The earlier transfinite contenders use explicit, linearly ordered ordinal
notations as atoms.  This module instead uses *addresses in a fixed
countably-branching well-founded tree*.  A level is a finite path from the
root; a level lies immediately below its parent exactly when that path is
valid in the tree.  Well-foundedness is inherited directly from structural
well-foundedness of `WTree`.

The particular tree `autonomousUniverseTree` contains `tree12` as branch zero
and, on the remaining branches, every finite autonomous hyper-closure above
`tree12`.  Thus the object language receives a genuinely ramified hierarchy
whose index is built from the W-language's own language-generated tree, rather
than from a fixed explicit notation system.
-/

namespace BusyLean.Ordinal

open BigNumber (WTree)

/-- A finite address in a countably branching tree.  The constructor is
"snoc-like": `child p i` appends branch `i` to the path `p`. -/
inductive TreePath : Type where
  | root : TreePath
  | child : TreePath → Nat → TreePath
  deriving DecidableEq, Repr

namespace TreePath

/-- Follow a finite address.  Invalid addresses return `none`. -/
def follow (t : WTree Nat) : TreePath → Option (WTree Nat)
  | .root => some t
  | .child p i =>
      match follow t p with
      | some (.node f) => some (f i)
      | _ => none

/-- Boolean validity of an address in a tree. -/
def valid (t : WTree Nat) (p : TreePath) : Bool :=
  match follow t p with
  | some _ => true
  | none => false

/-- `childLt t b a` says that `b` is a valid immediate child of `a` in `t`. -/
def childLt (t : WTree Nat) : TreePath → TreePath → Bool
  | .root, _ => false
  | .child p i, a => decide (p = a) && valid t (.child p i)

/-- Accessibility of a valid address, proved by structural recursion on the
subtree reached by that address. -/
theorem acc_follow (root : WTree Nat) :
    ∀ (cur : WTree Nat) (p : TreePath),
      follow root p = some cur →
      Acc (fun b a => childLt root b a = true) p
  | .leaf, p, hp => by
      refine Acc.intro p ?_
      intro b hb
      cases b with
      | root =>
          simp [childLt] at hb
      | child q i =>
          simp only [childLt, Bool.and_eq_true] at hb
```

```
          have hq : q = p := of_decide_eq_true hb.1
          subst q
          have hv : valid root (.child p i) = true := hb.2
          simp [valid, follow, hp] at hv
  | .node f, p, hp => by
      refine Acc.intro p ?_
      intro b hb
      cases b with
      | root =>
          simp [childLt] at hb
      | child q i =>
          simp only [childLt, Bool.and_eq_true] at hb
          have hq : q = p := of_decide_eq_true hb.1
          subst q
          exact acc_follow root (f i) (.child p i) (by
            simp [follow, hp])

/-- The immediate-valid-child relation is well-founded for every `WTree`. -/
theorem childLt_wf (root : WTree Nat) :
    WellFounded (fun b a => childLt root b a = true) := by
  refine ⟨fun p => ?_⟩
  cases hp : follow root p with
  | none =>
      refine Acc.intro p ?_
      intro b hb
      cases b with
      | root =>
          simp [childLt] at hb
      | child q i =>
          simp only [childLt, Bool.and_eq_true] at hb
          have hq : q = p := of_decide_eq_true hb.1
          subst q
          have hv : valid root (.child p i) = true := hb.2
          simp [valid, follow, hp] at hv
  | some cur =>
      exact acc_follow root cur p hp

/-- A simple increasing finite enumeration of paths.  Every finite path occurs
at some stage; duplicates are harmless for finite maxima. -/
def pathsUpTo : Nat → List TreePath
  | 0 => [.root]
  | n + 1 =>
      let ps := pathsUpTo n
      ps ++ ps.flatMap (fun p =>
        (List.range (n + 1)).map (fun i => .child p i))

end TreePath

/-- A fixed countable join containing `tree12` and all finite autonomous
hyper-closures above it.  Branch zero is exactly `tree12`. -/
def autonomousUniverseTree : WTree Nat :=
  .node fun k =>
    match k with
    | 0 => tree12
    | n + 1 =>
        Autonomous.realize
          (Autonomous.hyper n (Autonomous.encode tree12))

/-- Atoms for the ramified W-tree universe hierarchy. -/
abbrev WLevel := TreePath

instance : FL.AtomOrder WLevel where
  lt := TreePath.childLt autonomousUniverseTree

/-- Structural well-foundedness of the W-tree universe hierarchy. -/
theorem wLevel_wf : WellFounded (FL.ALt (α := WLevel)) :=
  TreePath.childLt_wf autonomousUniverseTree

```

```
/-- Valid W-tree levels of enumeration depth ‘n‘. -/
def wLevelsUpTo (n : Nat) : List WLevel :=
  (TreePath.pathsUpTo n).filter
    (TreePath.valid autonomousUniverseTree)

/-- The old language-generated index is visibly present at branch zero. -/
theorem follow_tree12 :
    TreePath.follow autonomousUniverseTree (.child .root 0) = some tree12 := rfl

end BusyLean.Ordinal
```

B Lean source: definition

Listing 3: `BusyLean/Contender20.lean`

```
import BusyLean.Ordinal.WTreeLadder
import BusyLean.Contender19

/-!
# Contender 20: autonomous W-tree universe reflection

‘‘‘
contender_20 := F20 (F20 42)
F20 n        := diagonalLift (schedule20 n) (rank20 n)
                  aTransformer orbit19 n n
‘‘‘

Contender 19 uses a linearly ordered ladder of explicit large-Veblen
notations.  Here the atoms are valid finite addresses in
‘Ordinal.autonomousUniverseTree‘, a countably branching well-founded tree that
contains the W-language-generated ‘tree12‘ and every finite autonomous
hyper-closure above it.  The resulting object language is a ramified tree of
predicative universes.  Its strong normalization is an immediate instance of
‘FL.sn_of_typed‘ over the generic well-founded universe ‘FL.wfUniv‘.

The seed family exposes the finite orbit of ‘F19‘: coordinate ‘i‘ contains the
‘(i+2)‘-fold iterate of ‘F19‘.  In particular coordinate zero is exactly
Contender 19’s family.  The schedule and diagonal rank are then self-fed from
an orbit whose length is itself ‘family19 n‘.

All recursion is structural.  The definition is computable in principle, but
it must never be evaluated at the contest input.
-/

namespace BusyLean.FL

open Ordinal

theorem sn_erase_wlevel {f : Nat → Nat} {fuel : Nat} {t : CTm WLevel}
    (h : infer fuel [] [] t = some Ty.nat) : F.SN f (erase t) :=
  sn_of_typed (wfUniv f wLevel_wf) (infer_sound fuel t (goodCtx_nil []) h).1

/-- The value of a ramified W-tree-universe program, type checked with ‘fuel‘,
relative to the oracle ‘f‘. -/
def avalue (f : Nat → Nat) (fuel : Nat) (t : CTm WLevel) : Nat :=
  match h : infer fuel [] [] t with
  | some A =>
      if hA : A = Ty.nat then
        F.decode (F.nf f ⟨erase t, sn_erase_wlevel (h.trans (congrArg some hA))⟩)
      else 0
  | none => 0

/-- Ramified W-tree-universe programs of depth ‘≤ n‘. -/
def autonomousLadderUpTo (n : Nat) : List (CTm WLevel) :=
```

```
  ctmsUpTo (wLevelsUpTo n) n

end BusyLean.FL

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- Reflection over the ramified W-tree universe hierarchy, with a countable
oracle family. -/
def reflectAFamily (fs : Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN (prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (FL.avalue (packFamily fs) n)
      (FL.autonomousLadderUpTo n))

/-- The family-aware ramified-universe reflection transformer. -/
def aTransformer : Transformer := fun fs _ n => reflectAFamily fs n

/-- Contender 19's construction as a function of its input:
`family19 42 = contender_19`. -/
def family19 (n : Nat) : Nat := F19 (F19 n)

/-- Finite iteration of `F19`, with the next iterate exposed at the outermost
constructor.  This orientation makes strict growth use only `input_lt_F19`;
no monotonicity of reflection is assumed. -/
def iterateF19 : Nat → Nat → Nat
  | 0, n => n
  | k + 1, n => F19 (iterateF19 k n)

/-- Coordinate `i` is the `(i+2)`-fold finite orbit of `F19` from `n`.
Coordinate zero is definitionally `family19 n`. -/
def orbit19 : Family := fun i n => iterateF19 (i + 2) n

/-- A self-fed orbit value used for the schedule and the lift rank. -/
def seedHeight20 (n : Nat) : Nat := orbit19 (family19 n) n

/-- The autonomous schedule is now built above the same W-tree that indexes the
object language, at a self-fed hyper level. -/
def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (seedHeight20 n + 42)
      (Autonomous.encode Ordinal.autonomousUniverseTree))

/-- Self-fed rank for diagonal hyperreflection. -/
def rank20 (n : Nat) : Nat := seedHeight20 n + 1

/-- Diagonal hyperreflection of the ramified W-tree universe language. -/
def F20 (n : Nat) : Nat :=
  diagonalLift (schedule20 n) (rank20 n)
    aTransformer orbit19 n n

/-- **Contender 20.** Never evaluate it. -/
def contender_20 : Nat := F20 (F20 42)

end BusyLean
```

C Lean source: comparison theorems

Listing 4: `BusyLean/Comparison20.lean`

```
import BusyLean.Contender20
import BusyLean.Comparison19

```

```
/-! # Contender 20: exact comparison theorems -/

namespace BusyLean

open BusyLean.W

/-- The additive guard in `reflectAFamily` strictly exceeds the finite maximum
it reflects. -/
theorem lt_reflectAFamily {fs : Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN (prefixMax (fun i => fs i n) n)
      (BigNumber.maximum (FL.avalue (packFamily fs) n)
        (FL.autonomousLadderUpTo n))) :
    m < reflectAFamily fs n := by
  show m < n + 1 + _
  have hM : ∀ M : Nat, M + 1 ≤ n + 1 + M := fun M => by
    rw [Nat.add_comm M 1]
    exact Nat.add_le_add_right (Nat.le_add_left 1 n) M
  exact Nat.lt_of_le_of_lt h
    (Nat.lt_of_lt_of_le (Nat.lt_succ_self _) (hM _))

/-- Every visible oracle coordinate lies below one autonomous-universe
reflection step. -/
theorem oracle_lt_reflectAFamily (fs : Family) {i n : Nat} (hi : i ≤ n) :
    fs i n < reflectAFamily fs n :=
  lt_reflectAFamily
    (Nat.le_trans (le_prefixMax (fun j => fs j n) hi)
      (BigNumber.le_maxN_left _ _))

/-- Every enumerated ramified-universe program lies below one reflection step. -/
theorem avalue_lt_reflectAFamily (fs : Family) {n : Nat}
    {t : FL.CTm Ordinal.WLevel} (ht : t ∈ FL.autonomousLadderUpTo n) :
    FL.avalue (packFamily fs) n t < reflectAFamily fs n :=
  lt_reflectAFamily
    (Nat.le_trans (le_maximum (FL.avalue (packFamily fs) n) ht)
      (BigNumber.le_maxN_right _ _))

/-- Autonomous-universe reflection is progressive. -/
theorem aTransformer_progressive : Progressive aTransformer :=
  fun seed _ _ _ hi => oracle_lt_reflectAFamily seed hi

/-- One more finite `F19` iterate is strictly larger.  No monotonicity of
`F19` is used. -/
theorem iterateF19_lt_succ (k n : Nat) :
    iterateF19 k n < iterateF19 (k + 1) n := by
  simpa only [iterateF19] using input_lt_F19 (iterateF19 k n)

/-- Coordinate zero of the seed orbit is exactly Contender 19's family. -/
theorem orbit19_zero (n : Nat) : orbit19 0 n = family19 n := rfl

/-- The exposed orbit is strictly increasing coordinate by coordinate. -/
theorem orbit19_lt_next (i n : Nat) : orbit19 i n < orbit19 (i + 1) n := by
  simpa only [orbit19, Nat.add_assoc, Nat.add_comm, Nat.add_left_comm]
    using iterateF19_lt_succ (i + 2) n

/-- Every seed-orbit coordinate visible at input `n` lies below `F20 n`.
This is a substantially stronger lower bound than merely embedding coordinate
zero. -/
theorem orbit19_lt_F20 {i n : Nat} (hi : i ≤ n) : orbit19 i n < F20 n := by
  have h :=
    ((diagonalLift_preservesProgressive (schedule20 n) (rank20 n))
      aTransformer aTransformer_progressive)
      orbit19 i n n hi
  simpa only [F20] using h

/-- At every input, `F20` strictly exceeds Contender 19's family. -/
theorem family19_lt_F20 (n : Nat) : family19 n < F20 n := by
  have h :=
    ((diagonalLift_preservesProgressive (schedule20 n) (rank20 n))
```

```
      aTransformer aTransformer_progressive)
      orbit19 0 n n (Nat.zero_le n)
  simpa only [orbit19_zero, F20] using h

/-- The raw autonomous-universe reflection step is strictly below the full
ranked hyperreflection. -/
theorem raw_reflection_lt_F20 (n : Nat) :
    reflectAFamily orbit19 n < F20 n :=
  (diagonalLift_strictlyInflationary (schedule20 n) (rank20 n))
    aTransformer aTransformer_progressive orbit19 n n n (Nat.le_refl n)

/-- Every ramified-universe program of depth at most `n`, run with the packed
seed orbit, is below `F20 n`. -/
theorem avalue_lt_F20 {n : Nat} {t : FL.CTm Ordinal.WLevel}
    (ht : t ∈ FL.autonomousLadderUpTo n) :
    FL.avalue (packFamily orbit19) n t < F20 n :=
  Nat.lt_trans (avalue_lt_reflectAFamily orbit19 ht)
    (raw_reflection_lt_F20 n)

/-- The input lies below Contender 19's family. -/
theorem input_lt_family19 (n : Nat) : n < family19 n :=
  Nat.lt_trans (input_lt_F19 n) (input_lt_F19 (F19 n))

/-- `F20` is strictly inflationary on numerical inputs. -/
theorem input_lt_F20 (n : Nat) : n < F20 n :=
  Nat.lt_trans (input_lt_family19 n) (family19_lt_F20 n)

/-- Pointwise, the old function `F19` is already below the new function. -/
theorem F19_lt_F20 (n : Nat) : F19 n < F20 n :=
  Nat.lt_trans (input_lt_F19 (F19 n)) (family19_lt_F20 n)

/-- The inner stage already dominates 44 finite iterates of `F19` from 42. -/
theorem iterateF19_44_42_lt_F20_42 : iterateF19 44 42 < F20 42 := by
  simpa only [orbit19] using
    (orbit19_lt_F20 (i := 42) (n := 42) (Nat.le_refl 42))

/-- A strong closed lower bound for the final contender: it exceeds an
`F19`-orbit whose length is itself `F20 42 + 2`. -/
theorem long_F19_orbit_lt_contender_20 :
    iterateF19 (F20 42 + 2) (F20 42) < contender_20 := by
  simpa only [orbit19, contender_20] using
    (orbit19_lt_F20 (i := F20 42) (n := F20 42) (Nat.le_refl _))

/-- **The required exact comparison:** Contender 19 is strictly smaller than
Contender 20. -/
theorem contender_19_lt_contender_20 : contender_19 < contender_20 :=
  Nat.lt_trans (family19_lt_F20 42) (input_lt_F20 (F20 42))

/-- The new contender also strictly exceeds every earlier accepted contender
by transitivity. -/
theorem contender_18_lt_contender_20 : contender_18 < contender_20 :=
  Nat.lt_trans contender_18_lt_contender_19 contender_19_lt_contender_20

end BusyLean
```

D Validation entry points

The archive also contains the complete smoke test, focused axiom/dependency audit, contest-style timing script, GitHub Actions workflow, integration patch, and recorded validation log. Keeping these as executable artifacts rather than reprinting them here avoids duplicating operational code in the article.

References

[Aczel 1978] Peter Aczel. The type theoretic interpretation of constructive set theory. In Angus Macintyre, Leszek Pacholski, and Jeff Paris, editors, *Logic Colloquium '77*, volume 96 of *Studies in Logic and the Foundations of Mathematics*, pages 55–66. North-Holland, 1978. doi:10.1016/S0049-237X(08)71989-X.

[Aczel 1986] Peter Aczel. The type theoretic interpretation of constructive set theory: Inductive definitions. In *Logic, Methodology and Philosophy of Science VII*, volume 114 of *Studies in Logic and the Foundations of Mathematics*, pages 17–49. Elsevier, 1986. doi:10.1016/S0049-237X(09)70683-4.

[Buchholz 1986] Wilfried Buchholz. A new system of proof-theoretic ordinal functions. *Annals of Pure and Applied Logic*, 32:195–207, 1986. doi:10.1016/0168-0072(86)90052-7.

[Rathjen 1999] Michael Rathjen. The realm of ordinal analysis. In S. Barry Cooper and John K. Truss, editors, *Sets and Proofs*, pages 219–280. Cambridge University Press, 1999. doi:10.1017/CBO9781107325944.011.

[Rathjen 2007] Michael Rathjen. The art of ordinal analysis. In *Proceedings of the International Congress of Mathematicians, Madrid 2006, Volume II*, pages 45–69. European Mathematical Society, 2007. doi:10.4171/022-2/3.

[Schütte 1977] Kurt Schütte. *Proof Theory*. Grundlehren der mathematischen Wissenschaften 225. Springer-Verlag, Berlin, Heidelberg, and New York, 1977. doi:10.1007/978-3-642-66473-1.